COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 / Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	09/01/2026	**DATE SUBMITTED:**	09/01/2026
NAMES:	Group 5 (Ramos, Reyes, Rodulfa, Salcedo, Santiago)	**INSTRUCTOR:**	Engr. Jimlord Quejardo

SEATWORK NO. #7.1 USING SORTING ALGORITHMS

1. PROCEDURE OUTPUT

QUESTIONS	ANSWERS
1. What is sorting algorithm?	Sorting is an algorithm that arranges the elements that were given in a list of particular order depending if it is ascending or descending.
2. Where can sorting algorithms be used?	It is used for searching algorithms, data management, database optimization, machine learning, data analysis, operating systems, etc.
3. Explain the different types of sorting algorithm.	Number of comparisons - also known as comparison-based sorting algorithms, it checks the elements of the list by key comparison operation. It checks for the best and worst case behavior. Examples of these are Quick Sort and Bubble Sort. Number of swaps - this method used to categorize by the number of swaps. Memory usage - used to create auxiliary locations to store and sort data temporarily. Recursion - this is also known as Divide and Conquer algorithm. Uses a self-referential function to call to break down into an array into smaller and simpler. An example of this is Merge Sort.
4. Give sample programs in C++ that uses sorting algorithms, specifically selection sort, insertion sort, and bubble sort. Explain how the programs work.	Selection sort: For selection sort, the program works by finding the minimum value repeatedly until the unsorted section is swapped to the corrected section. Insertion sort: For insertion sort, the program works by iterating it

	through an array and a sequence, it shifts the larger elements to the right and places it on each current element to its corrected position. Bubble sort: For bubble sort, the code uses an array to sort the unsorted integer using a temporary variable to replace it before putting it back to its designated position with the other elements in ascending order.

Selection sort:

```cpp
// C++ program for implementation of
// selection sort
#include <bits/stdc++.h>
using namespace std;

//Swap function
void swap(int *xp, int *yp)
{
    int temp = *xp;
    *xp = *yp;
    *yp = temp;
}

void selectionSort(int arr[], int n)
{
    int i, j, min_idx;

    // One by one move boundary of
    // unsorted subarray
    for (i = 0; i < n-1; i++)
    {

        // Find the minimum element in
        // unsorted array
        min_idx = i;
        for (j = i+1; j < n; j++)
        if (arr[j] < arr[min_idx])
            min_idx = j;

        // Swap the found minimum element
        // with the first element
        swap(&arr[min_idx], &arr[i]);
    }
}

//Function to print an array
void printArray(int arr[], int size)
{
    int i;
    for (i=0; i < size; i++)
        cout << arr[i] << " ";
    cout << endl;
}

// Driver program to test above functions
int main()
{
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr)/sizeof(arr[0]);
    selectionSort(arr, n);
    cout << "Sorted array: ";
    printArray(arr, n);
    return 0;
}
```

Insertion sort:

```cpp
// C++ program for insertion sort
#include <bits/stdc++.h>
using namespace std;

// Function to sort an array
// using insertion sort
void insertionSort(int arr[], int n)
{
    int i, key, j;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;

        // Move elements of arr[0..i-1],
        // that are greater than key, to one
        // position ahead of their
        // current position
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}

// A utility function to print
// an array of size n
void printArray(int arr[], int n)
{
    int i;
    for (i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

// Driver code
int main()
{
    int arr[] = { 12, 11, 13, 5, 6 };
    int N = sizeof(arr) / sizeof(arr[0]);

    insertionSort(arr, N);
    printArray(arr, N);

    return 0;
}
// This is code is contributed by rathbhupendra
```

Bubble sort:

```cpp
// Bubble sort in C++

#include <iostream>
using namespace std;

// perform bubble sort
void bubbleSort(int array[], int size) {

  // loop to access each array element
  for (int step = 0; step < size -1; ++step) {

    // loop to compare array elements
    for (int i = 0; i < size - step - 1; ++i) {

      // compare two adjacent elements
      // change > to < to sort in descending order
      if (array[i] > array[i + 1]) {

        // swapping elements if elements
        // are not in the intended order
        int temp = array[i];
        array[i] = array[i + 1];
        array[i + 1] = temp;
      }
    }
  }
}
```

```cpp
// print array
void printArray(int array[], int size) {
  for (int i = 0; i < size; ++i) {
    cout << "  " << array[i];
  }
  cout << "\n";
}

int main() {
  int data[] = {-2, 45, 0, 11, -9};

  // find array's length
  int size = sizeof(data) / sizeof(data[0]);

  bubbleSort(data, size);

  cout << "Sorted Array in Ascending Order:\n";
  printArray(data, size);
}
```

References:

GeeksforGeeks. (2021, November 23). *Classification of sorting algorithms*. GeeksforGeeks.

https://www.geeksforgeeks.org/dsa/classification-of-sorting-algorithms/

GeeksforGeeks. (2026, January 20). *Applications, Advantages and Disadvantages of sorting algorithm.* GeeksforGeeks. https://www.geeksforgeeks.org/dsa/applications-advantages-and-disadvantages-of-sorting-algorithm/

GeeksforGeeks. (2025, July 23). *C++ Program for Selection Sort.* GeeksforGeeks. https://www.geeksforgeeks.org/cpp/cpp-program-for-selection-sort/

GeeksforGeeks. (2025, July 23). *C++ Program for Insertion Sort.* GeeksforGeeks. https://www.geeksforgeeks.org/cpp/cpp-program-for-insertion-sort/

Programiz. *Bubble Sort.* Programiz. https://www.programiz.com/dsa/bubble-sort

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Finding references**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used Gemini to find references and anything that I type here is purely on my own.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.